A PRELIMINARY REPORT ON

EFFICIENT LOSSLESS FILE SYSTEM

SUBMITTED TO THE SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE
IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE

OF

BACHELOR OF ENGINEERING
(COMPUTER ENGINEERING)

SUBMITTED BY

SHRUTI NIKAM	Exam No: **B1908804282**
KOMAL PATIL	Exam No: **B1908804289**
ADVAIT PANDHARPURKAR	Exam No: **B1908804284**
ATHARVA WAGHCHOURE	Exam No: **B1908804328**

Under the Guidance of
Prof. Amruta Chitari

AJEENKYA
DY Patil School of Engineering

DEPARTMENT OF COMPUTER ENGINEERING

AJEENKYA DY PATIL SCHOOL OF ENGINEERING
CHARHOLI BUDRUK, LOHEGAON, PUNE 412105

SAVITRIBAI PHULE PUNE UNIVERSITY
2024 -2025

AJEENKYA
DY Patil School of Engineering

AJEENKYA DY PATIL SCHOOL OF ENGINEERING
DEPARTMENT OF COMPUTER ENGINEERING

CERTIFICATE

This is to certify that the Project Report Entitled

" EFFICIENT LOSSLESS FILE SYSTEM"

Submitted by

SHRUTI NIKAM	Exam No: **B1908804282**
KOMAL PATIL	Exam No: **B1908804289**
ADVAIT PANDHARPURKAR	Exam No: **B1908804284**
ATHARVA WAGHCHOURE	Exam No: **B1908804328**

is a bonafide student of this institute and the work has been carried out by him/her under the supervision of **Prof. Amruta Chitari** and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University, for the award of the degree of **Bachelor of Engineering** in Computer Engineering.

Prof. Amruta Chitari
Guide
Department of Computer Engineering

Dr. Pankaj Agarkar
H.O.D
Department of Computer Engineering

Signature of Internal Examiner:

Signature of External Examiner:

Dr. F. B .Sayyad
Principal
AJEENKYA DY PATIL SCHOOL OF ENGINEERING , Lohegaon, Pune-412015

Place : Pune
Date :

ACKNOWLEDGEMENT

It gives me a great pleasure and immense satisfaction to present this special topic of project report on **"Efficient Lossless File System"** which is the result of unwavering support, expert guidance and focused direction of guide **Prof. Amruta Chitari** to whom I express my deep sense of gratitude and humble thanks, for his valuable guidance throughout the presentation work.

Furthermore , I am indebted to Dr. Pankaj Agarkar , HOD Computer Engineering and Dr. F. B. Sayyad, Principal whose constant encouragement and motivation inspired me to do my best.

Last but not the least I sincerely thank to my colleagues, the staff and all others who directly or indirectly helped us and made numerous suggestions which have surely improved the quality of my work.

ADVAIT PANDHARPURKAR

KOMAL PATIL

SHRUTI NIKAM

ATHARVA WAGHCHOURE

ABSTRACT

In today's digital landscape, the exponential growth of data presents significant challenges for effective storage and management. Traditional file systems often face limitations in storage efficiency and retrieval speed, leading to increased operational costs and slower workflows. This project addresses these challenges by introducing an innovative lossless file system designed to optimize data storage while maintaining data integrity.

The system leverages advanced compression algorithms that significantly reduce the required disk space without sacrificing the quality of the stored data. Additionally, its modular architecture allows for seamless integration with various storage technologies, ensuring compatibility and flexibility across different environments. The implementation of efficient indexing methods further enhances retrieval times, enabling quick access to large datasets.

Performance evaluations demonstrate that our lossless file system not only improves storage efficiency but also accelerates data retrieval speeds compared to conventional file systems. The findings indicate that organizations can achieve considerable reductions in storage costs while enhancing overall operational efficiency.

In conclusion, this project presents a comprehensive solution to modern data management issues, positioning the efficient lossless file system as a valuable tool for businesses and institutions looking to adapt to the increasing demands of data storage and access in a digital-first world.

TABLE OF CONTENTS

CHAPTER **TITLE** **PAGE NO.**

Sr.No.	Title of Chapter	Page No.
01	**Introduction**	9
1.1	Motivation of the Project	10
1.2	Literature Survey	11
02	**Problem Definition & Scope**	13
2.1	Problem Statement	13
2.2	Goal & Objective	14
2.3	Statement of Scope	14
2.4	Methodolgies of Problem Solving and Efficiency issues	15
03	**Project Plan**	16
3.1	Project Estimates	16
3.2	Risk Management w.r.t NPHard Analysis	17
3.3	Project Schedule	18
04	**Software Requirements Specification**	19
4.1	Database Requirements	19
4.2	Software Requirements(Platform Choice)	20
4.3	Hardware Requirements	21
4.4	Analysis Models: SDLC Model to be applied	22
05	**System Design**	23
5.1	System Architecture	23
5.2	Mathematical Model	25
5.3	Data Flow Diagrams	26
5.4	Use case Diagrams	27
5.5	Sequence Diagrams	28
06	**Other Specification**	29
6.1	Advantages	29
6.2	Limitations	29
6.3	Applications	30

07	**Conclusions & Future Work**	31
	Appendix A: Problem statement feasibility assessment using, satisfiability analysis and NP Hard,NP-Complete or P type using modern algebra and relevant mathematical models. **Appendix B:** Details of the papers referred in IEEE format (given earlier) Summary of the above paper in not more than 3-4 lines. Here you should write the seed idea of the papers you had referred for preparation of this project report in the following format. Example: Thomas Noltey, Hans Hanssony, Lucia Lo Belloz,"Communication Buses for Automotive Applications" In *Proceedings of the* 3rd *Information Survivability Workshop (ISW-2007),* Boston, Massachusetts, USA, October 2007. IEEE Computer Society. **Appendix C:** Plagiarism Report	
	References	32

LIST OF ABBREVATIONS

ABBREVIATION	ILLUSTRATION
ELFS	Efficient Lossless File System
BTRFS	B-Tree File System
ZStd	Zstandard Algorithm
LZMA	Lempel–Ziv–Markov Algorithm

LIST OF FIGURES

FIGURE	ILLUSTRATION	PAGE NO.
5.1	System Architecture	25
5.2	Mathematical Model	27
5.3	Data Flow Diagrams	28
5.4	Use case Diagram	29
5.5	Sequence Diagram	30

1. INTRODUCTION

ELFS (Efficient Lossless File System) is designed to store data with both high efficiency and data integrity. Being lossless, ELFS ensures that any data stored can be retrieved without alteration, making it suitable for applications where data accuracy is essential. Built as a read-write file system, ELFS supports both data retrieval and storage, which enhances its versatility for general-purpose storage needs. Drawing inspiration from compressed file systems like DwarFS and SquashFS, ELFS optimizes space by leveraging modern compression techniques. It specifically uses the Zstandard (Zstd) compression algorithm, which provides a balanced approach to compression efficiency and speed, allowing data to be stored compactly without significantly impacting access times. An innovative aspect of ELFS is its dual-storage approach with "hot" and "cold" pots. Data that needs to be accessed frequently is stored in the hot pot in a decompressed state, providing faster access. In contrast, infrequently accessed data resides in the cold pot in a compressed format, optimizing storage space. By utilizing these features, ELFS effectively balances data accessibility, space savings, and speed, making it a powerful and flexible file system solution.

ELFS can apply varying levels of compression based on file types, access patterns, or user preferences. This flexibility allows higher compression for less critical or infrequently accessed data and lighter compression or no compression for essential, frequently accessed files. ELFS is designed to be scalable, making it suitable for a variety of environments, from small devices to large, enterprise-level storage systems. Its efficient structure allows it to manage both small files and large data sets effectively.

1.1. MOTIVATION OF THE PROJECT

The motivation behind the Efficient Loss

less File System (ELFS) project is to develop a file system that optimizes data storage while ensuring no loss of information. In modern systems, where data storage demands are rapidly increasing, there is a need for a solution that compresses files effectively without compromising data integrity. ELFS aims to improve storage efficiency, reduce redundant data, and speed up file access, while providing robust support for large-scale data storage systems. This ensures that users can store more data in less space, making the system highly scalable and cost-effective.

1.2.LITERATURE SURVEY

Base paper used for our research includes:

[1] Xiang Gao1, Mingkai Dong2, Xie Miao1, Wei Du1, Chao Yu1, and Haibo Chen2,1 , EROFS: A Compression-friendly read only file system for resource-scare devices: It introduces a file system designed for resource-constrained devices. With the surge in data and the prevalence of low-power devices. Overall, EROFS represents a significant advancement in file system technology for limited-resource environments.

[2] Josef Bacik, Chris Mason FusionIO , IBM Research Report: The IBM Research Report on compression focuses on advanced techniques to enhance data storage efficiency as data volumes increase. It examines various compression algorithms, emphasizing the need for a balance between compression ratios and processing speed.Overall, it highlights IBM's commitment to developing compression technologies that reduce storage costs and boost system performance.

[3] Sanjay Ghemawat, Howard Gobioff, and Shun-Tak Leung Google , The Google File System: The Google File System (GFS) compression base paper presents GFS as a scalable distributed file system designed for large data sets. It focuses on integrated compression techniques that optimize storage efficiency while ensuring high performance for data access. Overall, it showcases GFS as a leading solution in managing vast amounts of data efficiently.

[4] Yann Collet ,2019 , Zstandard : A Fast lossless compressison algorithm : Zstandard (Zstd) is a fast lossless compression algorithm developed by Facebook, known for its high compression ratios and speed. It is designed for both efficient compression and rapid decompression, making it ideal for various applications, including real-time data processing. Zstandard is a robust solution for modern data compression needs.

[5] Di wu,Zizhong Chen,Franck Cappello , Efficient lossless compression of numerical data arrays: This paper focuses on techniques for efficiently compressing numerical data arrays without loss of information. It explores innovative algorithms designed to optimize

storage and enhance data retrieval speeds. Overall, the paper highlights the importance of efficient lossless compression in managing large numerical datasets effectively.

2. PROBLEM DEFINITION & SCOPE

Problem Statement

As data storage needs grow exponentially, current file systems struggle to balance storage efficiency and data integrity. Existing solutions often either compromise file compression quality or introduce data loss. The Efficient Lossless File System (ELFS) seeks to address this by providing a file system that compresses data without any loss of information, improving storage utilization while ensuring quick access and maintaining data accuracy. ELFS aims to optimize storage in environments with limited resources, ensuring efficient space management and performance without sacrificing data fidelity.

2.1. Goal &Objective

. The goal of the Efficient Lossless File System (ELFS) is to deliver a high-efficiency storage solution that can store and manage large data volumes while ensuring data integrity. Being **lossless**, ELFS guarantees that all data can be retrieved exactly as it was stored, without any degradation, making it suitable for environments where data accuracy is essential. By using compression techniques (like Zstandard) and **dual storage layers** (hot and cold pots), ELFS optimizes storage space: frequently accessed data is kept in an easily accessible format, while less-used data is stored in a compressed format to save space. This setup allows ELFS to handle substantial data loads reliably and cost-effectively, supporting quick access and effective space utilization.

The objectives include:

a. Implementing highly compressed writable filesystem.
b. Cold and hot storage pots for file io operations.
c. Data agnostic filesystem for wide use case.
d. Data integrity locks for concurrent file operations.

2.2. Statement of Scope

The scope of the Efficient Lossless File System (ELFS) encompasses the development of a file system that focuses on lossless data compression, efficient storage management, and fast data retrieval. ELFS is designed to handle a wide range of file types, ensuring that no data is lost during compression. It is applicable to both personal and enterprise-level storage systems, providing scalability for growing data needs. The system will also support integrity checks to ensure complete data restoration and seamless access to compressed files across various platforms and environments.

2.3. Methologies of Problem Solving and Efficient Issues

To solve the challenges of ELFS, the methodology involves using advanced lossless compression algorithms to reduce file sizes while preserving data integrity. Key steps include:

- Algorithm Selection: Employing efficient compression techniques (like Huffman or Lempel-Ziv) that guarantee no loss of data.
- Data Structure Optimization: Using optimized data structures for fast storage and retrieval.
- Testing and Validation: Rigorously testing the system to ensure that all files can be fully decompressed without errors.

For addressing efficiency issues:

- Memory Management: Implementing optimized memory usage to handle large files without slowing down the system.
- Compression Speed: Balancing the trade-off between compression efficiency and processing time to ensure fast file access.
- Scalability: Ensuring the file system performs well under increasing data loads, maintaining performance across small and large storage environments.

3. PROJECT PLAN

3.1. PROJECT ESTIMATES

The ELFS project estimates focus on the time, resources, and cost needed for successful implementation.

- Time Estimate:

Development: 5-6 months for coding, testing, and integration.

Testing & Debugging: 1-2 months to ensure stability and performance.

- Resource Estimate:

Team:4 developers with expertise in file systems and compression algorithms.

Tools: Software development tools, testing environments, and storage hardware.

- Cost Estimate:

Budget: Allocated for personnel, software licenses, hardware, and testing resources. It needs Rs 0 cost to build the project.

These aim to ensure that the project is completed efficiently within the allotted budget and time frame.

3.2. RISK MANAGEMENT w.r.t NP-HARD ANALYSIS

In developing ELFS, there is a potential risk of encountering NP-Hard problems, particularly in optimizing file compression and storage efficiency.

Risk Management Strategies:

- Algorithm Selection: Use heuristic or approximation algorithms that provide near-optimal solutions in a reasonable time, avoiding NP-Hard bottlenecks in real-time compression.
- Modular Design: Implement a flexible, modular system that allows swapping out inefficient components or algorithms without affecting the entire system.
- Parallel Processing: Mitigate performance risks by leveraging parallel processing techniques to speed up tasks that could become computationally expensive.
- Performance Monitoring: Continuously monitor system performance to identify and address inefficiencies early.

By proactively managing these risks, the project can avoid performance slowdowns due to NP-Hard problems while maintaining system efficiency.

3.3. PROJECT SCHEDULE

The project schedule for ELFS is designed to ensure timely completion within the allocated time frame.

- Phase 1: Planning & Requirement Analysis (2 weeks)
  Define system requirements, compression algorithms, and architecture.
- Phase 2: Design & Architecture (2 weeks)
  Develop system architecture, database structure, and file handling mechanisms.
- Phase 3: Development (4 weeks)
  Implement compression algorithms, file system features, and user interface.
- Phase 4: Testing & Debugging (8 weeks)
  Perform unit, integration, and performance testing to ensure system stability and accuracy.
- Phase 5: Optimization & Refinement (4 weeks)
  Fine-tune algorithms and optimize performance for scalability and efficiency.
- Phase 6: Deployment & Maintenance (Ongoing)
  Deploy the system and provide continuous support and updates.
  This schedule ensures structured progress while minimizing delays, focusing on efficient resource use and timely delivery.

4. SOFTWARE REQUIREMENTS SPECIFICATION

4.1. DATABASE REQUIREMENTS

The database requirements for ELFS focus on managing file metadata, compression details, and user information effectively.

- Storage of File Metadata:

Track file names, sizes, types, compression ratios, and locations.

Maintain records of file access, modification, and creation dates.

- Compression Details:

Store information about the compression algorithm used, original size, and compressed size.

Support for lossless decompression mapping to ensure data integrity.

- User and Access Management:

Manage user accounts, permissions, and roles for secure file access.

Implement logging to track user actions and system interactions.

- Scalability and Performance:

The database should scale efficiently to handle large amounts of metadata.

Optimize for quick retrieval and storage operations, especially in large-scale systems.

This ensures the system efficiently manages data while supporting high performance and integrity.

4.2. SOFTWARE REQUIREMENTS(platform choice)

The software requirements for ELFS focus on selecting a platform that supports efficient development, performance, and scalability.

- Operating System:

Linux-based systems (Arch Linux) for better file system management, security, flexibility.
Cross-platform compatibility (Windows, macOS) for broader user support.

- Programming Language:

Rust programming language is used primarily for its safety and performance features.

- Compression Libraries:

Zstd for integrating proven lossless compression algorithms.

This platform choice ensures that ELFS is efficient, scalable, and adaptable across different environments.

4.3. HARDWARE REQUIREMENTS

The hardware requirements for the Efficient Lossless File System (ELFS) focus on ensuring optimal performance, storage capacity, and reliability.

- Server Specifications:

Processor: Multi-core CPU (Intel 13th Gen Ultra 9) for handling concurrent file operations and compression tasks.

Memory: 32 GB DDR5

- Storage:

Hard Drives: SSDs for faster read/write speeds and reduced latency, particularly beneficial for high-frequency file access.

Capacity: Sufficient storage space (e.g., 1 TB or more) to accommodate large datasets and user files.

- Networking:

Network Interface: High-speed network interface cards (NIC) to ensure quick data transfers, especially in multi-user environments.

- Backup Solutions:

Redundant Array of Independent Disks (RAID): Implement RAID configurations for data redundancy and reliability.

- Backup Storage: External or cloud storage options for regular data backups and recovery.

These hardware specifications will help ELFS maintain high performance, reliability, and scalability in handling lossless file operations.

4.4. ANALYSIS MODELS: SDLC MODEL TO BE APPLIED

The Software Development Life Cycle (SDLC) model suitable for the Efficient Lossless File System (ELFS) is the Agile Model. This model is chosen for its flexibility and iterative approach, which aligns well with the dynamic requirements of developing a file system. Key Features of the Agile Model:

- Iterative Development:

Development is divided into small, manageable increments (sprints), allowing for regular feedback and adjustments based on user needs.

- Collaboration:

Encourages close collaboration between developers, testers, and stakeholders to ensure that requirements are met effectively.

- Continuous Testing:

Regular testing is integrated into the development process, enabling early detection of issues and ensuring a robust final product.

- Adaptability:

The Agile Model allows for quick adaptation to changing requirements, which is crucial for optimizing file system performance and addressing new challenges.

- User Feedback:

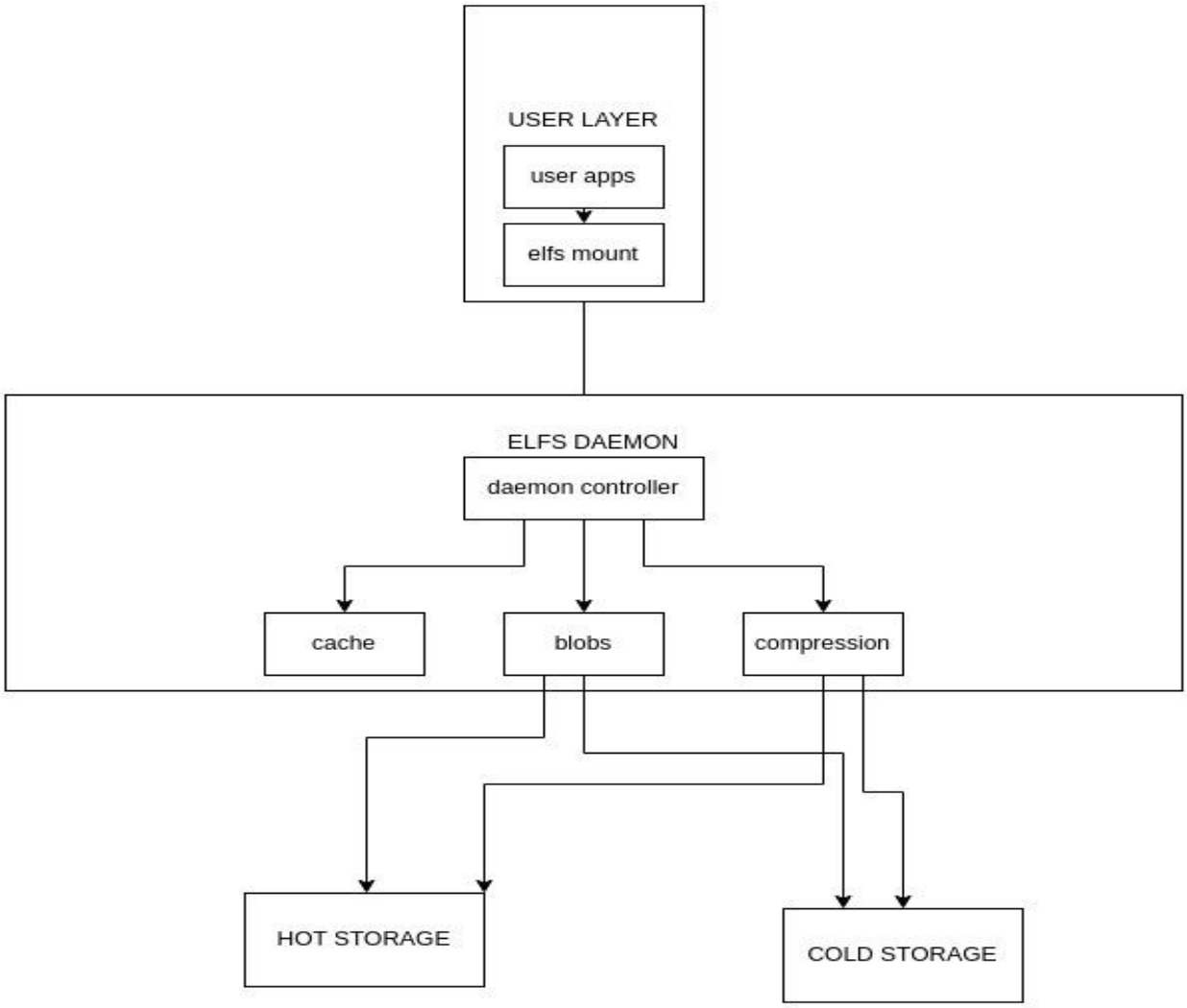
Involves users early and often to gather feedback, ensuring that the final product aligns with user expectations and needs.

This SDLC model promotes a responsive development environment that is ideal for creating the ELFS, ensuring high quality and efficiency throughout the project.

5. SYSTEM DESIGN

5.1.SYSTEM ARCHITECTURE

- Compression: Utilizes Zstd for efficient file compression.
- Read-Write Support: Enables both reading and writing of data.
- Storage Pots: Manages data in hot (decompressed) and cold (compressed) storage.



- Removes duplicate data blocks to optimize storage by only storing unique data.
- Uses Zstandard (Zstd) compression for efficient, fast compression and decompression, balancing space savings with speed.
- Dual Storage Layers:

  Hot Pot: Stores frequently accessed data in decompressed form for fast access.

  Cold Pot: Holds infrequently accessed data in compressed form to save space.

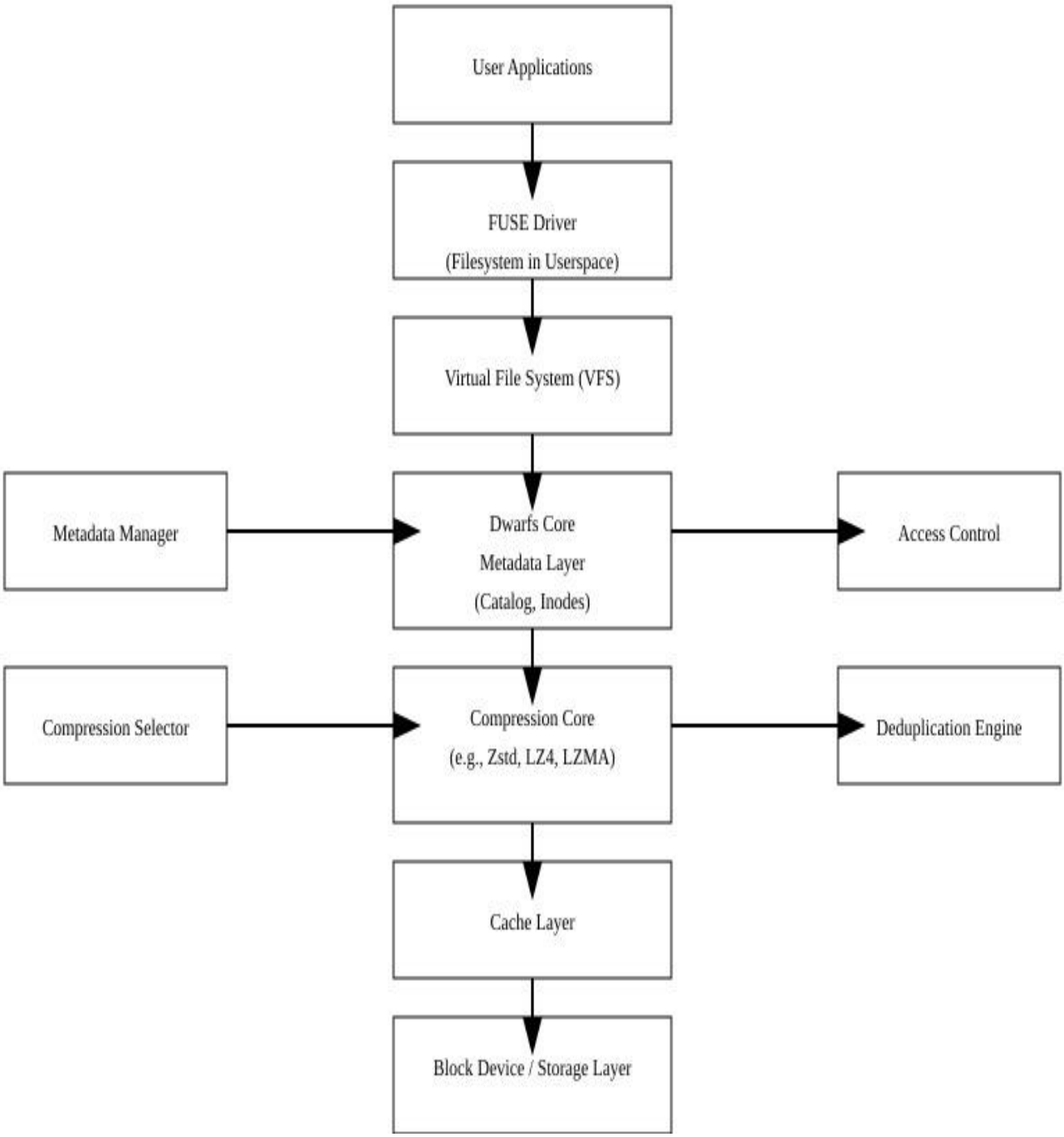
- Tools:

  a. **elfs-mount**: Mounts an ELFS file system, making its files accessible within a specified directory. This allows users to view, edit, or add files as if it were a regular storage area.
  b. **elfs-extract**: Extracts specific files or directories from an ELFS file system without mounting it. This is useful for accessing or backing up selected data without loading the full system.
  c. **elfs-make**: Creates a new ELFS file system, initializing its storage structure and setting options like compression or encryption. This tool is essential for setting up ELFS on new storage.
  d. **elfscat**: Displays the contents of files within an ELFS file system, decompressing them if needed. It provides a quick way to view file contents without extraction or mounting.

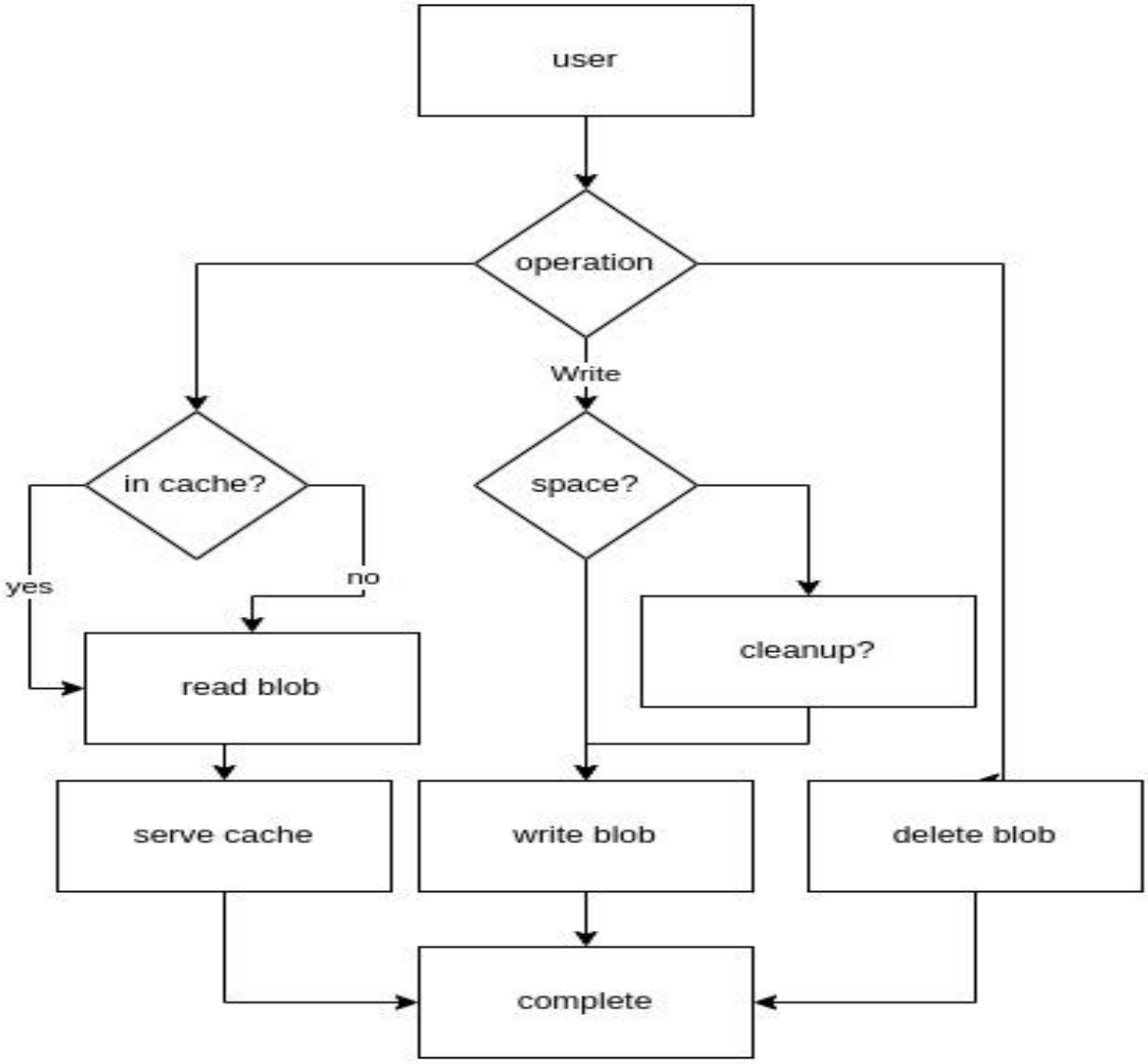
5.2.MATHEMATICAL MODEL

ALGORITHM	TIME COMPRESSION	COMMAND ITESELF	SIZE OF ARCHIVE
Pzstd(highest compression)	~15min	Pzstd ram.tar –o ram.tar.zst –p16 –ultra -22	9.3G
bz	21min	Time tar cjf file.core.tar.bz2 file.core	10G
lzma	1hr 20 min	Time tar cJf file.core.tar.xz file.core	9.8G
7zip	~6min	Time pigz –c –k -9 file.core|7z a –si –m0=lzma2 –mx=9 –mfb=64 –md=32	9.8G
gzip	~3min	Time pigz –c –k -9 file.core|zip file.core.zip-	9.9G
Zstd (integrated with RamSnapper)	~1min		9.8G

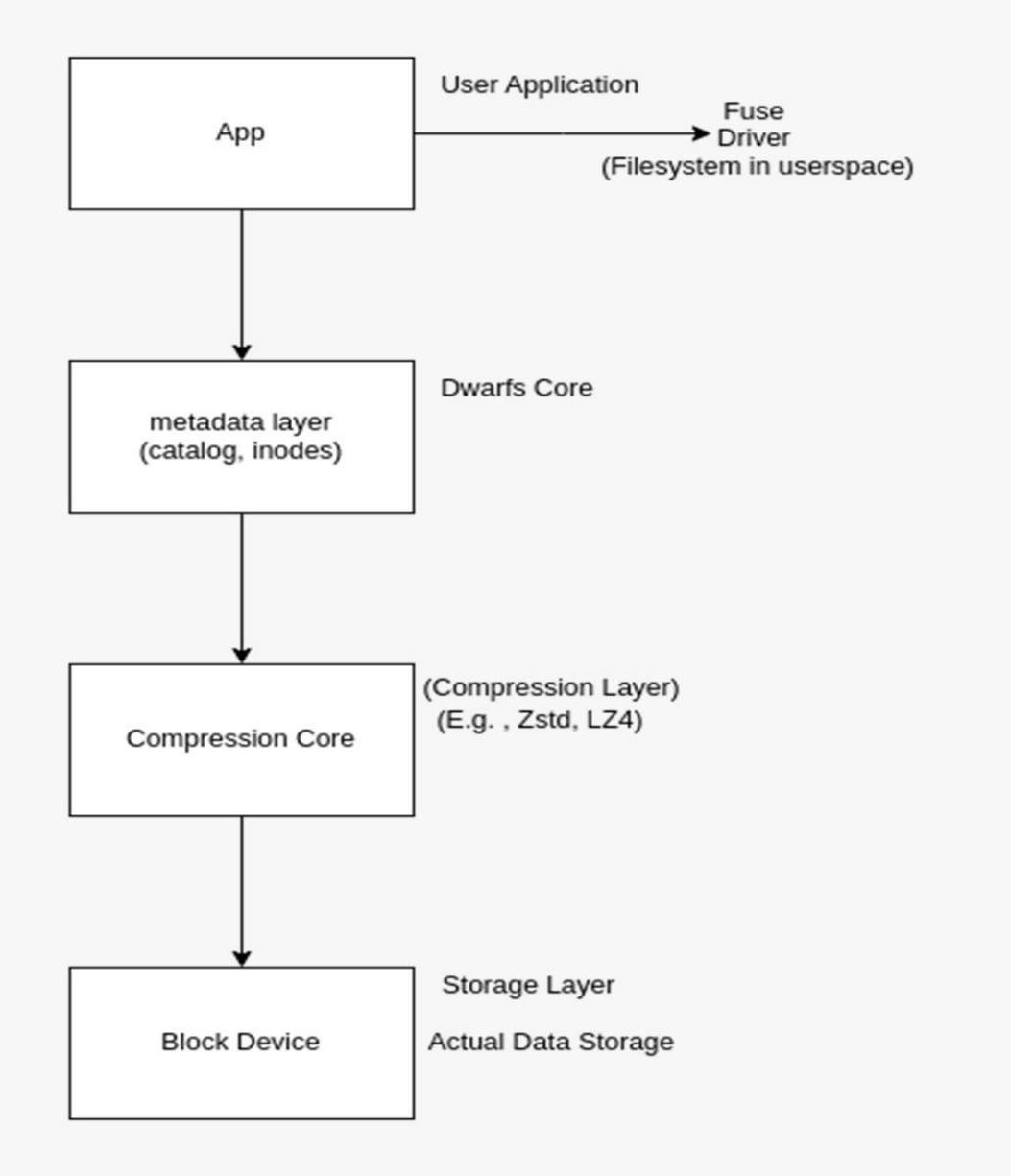
5.3.DATA FLOW DIAGRAMS



5.4.USE CASE DIAGRAM



5.5.SEQUENCE DIAGRAM



6. OTHER SPECIFICATION

6.1. ADVANTAGES

The Efficient Lossless File System (ELFS) offers several key advantages that make it a valuable tool in various sectors. It ensures data integrity by preserving files in their original form, making it easier to detect unauthorized modifications or plagiarism.It enables quick retrieval and processing of files. Also provides data agnostic filesystem for wide use case. ELFS uses lossless compression, reducing file sizes without affecting quality, which helps save storage space. Another benefit is its ability to track the origin and changes of files, providing a clear history for verifying content authenticity. The system also enhances security by securing file sharing with traceable metadata, allowing for the detection of unauthorized copying. Additionally, ELFS supports integration with anti-plagiarism tools, enabling efficient and accurate content comparison. Its scalability makes it ideal for managing large datasets, especially in industries where maintaining originality and data integrity is crucial, such as education and research.

6.2. LIMITATIONS

The Efficient Lossless File System (ELFS) has several disadvantages that organizations should consider. One significant challenge is the complex implementation process, which may require specialized technical expertise that not all organizations possess. Additionally, the focus on lossless compression can result in slower performance when accessing large datasets, potentially hindering user experience. The requirement for additional metadata to track file origins and modifications can lead to increased storage overhead compared to simpler file systems. Compatibility issues may arise, as ELFS might not integrate seamlessly with all existing software and systems, necessitating costly adjustments. Moreover, the initial setup and training can be expensive, posing a barrier for smaller organizations. Lastly, despite its security features, ELFS could still be susceptible to misuse if not properly managed, allowing users to manipulate file histories and undermine the system's integrit.

6.3. APPLICATIONS

- Data Archiving: Efficiently stores large volumes of data while maintaining integrity for long-term access.
- Cloud Storage: Provides lossless compression for files uploaded to cloud services, optimizing space and bandwidth.
- Multimedia Storage: Manages and compresses images, videos, and audio files without quality loss, ideal for media libraries.
- Database Management: Stores database backups and logs efficiently, ensuring quick access and reduced storage costs.
- Software Development: Facilitates version control systems by efficiently managing source code and binaries.
- Research Data Management: Preserves research data with high fidelity, enabling easy sharing and collaboration among researchers.

7. CONCLUSIONS & FUTURE WORK

In conclusion, the Efficient Lossless File System (ELFS) offers significant potential in the fight against plagiarism by providing secure, traceable, and authentic digital content management. Its ability to preserve data integrity, track file histories, and enhance the accuracy of anti-plagiarism tools makes it a valuable asset in education, research, and other fields where originality is critical. As ELFS technology evolves, it could play a crucial role in safeguarding intellectual property and ensuring the credibility of digital content.

The future scope of the Efficient Lossless File System (ELFS) in reducing plagiarism is promising. ELFS, with its ability to maintain data integrity, ensures that files remain unchanged, making it easier to detect unauthorized modifications or duplicated content. This feature could be invaluable in fields like education and research, where originality is critical. By tracking the origin and modification history of files, ELFS can help verify the authenticity of digital documents. Additionally, its secure file-sharing capabilities allow for traceable metadata, ensuring that any attempt to plagiarize content can be easily flagged. In combination with anti-plagiarism tools, ELFS can also efficiently store and index large datasets for accurate comparison, further enhancing plagiarism detection. As such, ELFS holds significant potential in combating plagiarism by ensuring secure, traceable, and original content in various sectors.

8. REFERENCES

[1] L. A. González, G. J. Bishop-Hurley, R. N. Handcock, and C. Crossman, "Behavioral classification of data from collars containing motion sensors in grazing cattle," Compute. Electron. Agric., no. 110, pp. 91-102, 2015.

[2] BENAVIDES, T., TREON, J., HULBERT, J., AND CHANG, W. The enabling of an Execute-In-Place architecture to reduce the embedded system memory footprint and boot time. JCP 3, 1 (2008), 79-89.

[3] Frank Schmuck and Roger Haskin. GPFS: A shared-disk file system for large computing clusters. In Proceedings of the First USENIX Conference on File and Storage Technologies, pages 231-244, Monterey, California, January 2002.

[4] Sanjay Ghemawat, Howard Gobioff, and Shun-Tak Leung Google," The Google File System".

[5] Xiang Gao1, Mingkai Dong2, Xie Miao1, Wei Du1, Chao Yu1, and Haibo Chen2,1 1Huawei Technologies Co., Ltd. 2Shanghai Jiao Tong University," EROFS: A Compression-friendly Readonly File System for Resource-scarce Devices".

[6] Ohad Rodeh, Josef Bacik, Chris Mason FusionIO , IBM Research Division Almaden Research Center 650 Harry Road San Jose, CA 95120-6099 USA.

[7] CHEN,R.,WANG,Y.,HU,J.,LIU,D.,SHAO,Z.,AND GUAN, Y. Unified non-volatile memory and NAND flash memory architecture in smartphones. In Design Automation Conference (ASP-DAC), 2015 20th Asia and South Pacific (2015), IEEE, pp. 340–345.

[8] HYUN, S., BAHN, H., AND KOH, K. Lecramfs: an efficient compressed file system for flash-based portable consumer devices. IEEE Transactions on Consumer Electronics 53 (2007).

[9] JEONG, S., LEE, K., HWANG, J., LEE, S., AND WON,Y. AndroStep: Android storage

performance analysis tool. In Software Engineering (Workshops) (2013),vol. 13, pp. 327–340.

[10] JEONG, S., LEE, K., HWANG, J., LEE, S., AND WON,Y. Framework for analyzing android I/O stack behavior:from generating the workload to analyzing the trace. Future Internet 5, 4 (2013), 591–610.

[11] John Howard, Michael Kazar, Sherri Menees, David Nichols, Mahadev Satyanarayanan, Robert Sidebotham, and Michael West. Scale and performance in a distributed file system. ACM Transactions on Computer Systems, 6(1):51–81, February 1988.

[12] Barbara Liskov, Sanjay Ghemawat, Robert Gruber, Paul Johnson, Liuba Shrira, and Michael Williams.Replication in the Harp file system. In 13th Symposium on Operating System Principles, pages 226–238, Pacific Grove, CA, October 1991.

[13] David A. Patterson, Garth A. Gibson, and Randy H.Katz. A case for redundant arrays of inexpensive disks (RAID). In Proceedings of the 1988 ACM SIGMOD International Conference on Management of Data, pages 109–116, Chicago, Illinois, September 1988.

[14] Steven R. Soltis, Thomas M. Ruwart, and Matthew T.O'Keefe. The Gobal File System. In Proceedings of the Fifth NASA Goddard Space Flight Center Conference on Mass Storage Systems and Technologies, College Park, Maryland, September 1996.

[15] Chandramohan A. Thekkath, Timothy Mann, and Edward K. Lee. Frangipani: A scalable distributed file system. In Proceedings of the 16th ACM Symposium on Operating System Principles, pages 224–237,Saint-Malo, France, October 1997.